UML-Klassendiagramme mit TikZ erstellen

Fortbildungsmaterial für MINT-Lehrkräfte

25. Mai 2025

UML-Klassendiagramme sind ein zentrales Werkzeug in der objektorientierten Programmierung und Softwareentwicklung. Sie visualisieren die Struktur von Klassen, deren Attribute, Methoden und Beziehungen zueinander. Mit TikZ und benutzerdefinierten Stilen können MINT-Lehrkräfte professionelle UML-Klassendiagramme effizient erstellen. Diese Handreichung zeigt, wie Sie mit eigenen TikZ-Stilen und -Befehlen wartbare und professionelle UML-Diagramme entwickeln können.

1 Grundlagen und Stil-Definitionen

TikZ ist in jeder LaTeX-Installation verfügbar und bietet durch seine Flexibilität die perfekte Grundlage für die Erstellung professioneller UML-Diagramme. Für effiziente und konsistente Diagrammerstellung definieren wir zunächst eigene Stile, die alle wichtigen UML-Elemente abdecken und eine einheitliche Formatierung gewährleisten.
Für die professionelle Erstellung von UML-Diagrammen benötigen wir neben dem Grundpaket TikZ einige spezialisierte Bibliotheken, die verschiedene Funktionalitäten bereitstellen:

```
\usepackage{tikz}
\usetikzlibrary{positioning, fit, calc, shapes.multipart, arrows.meta}
```

Diese Bibliotheken erfüllen jeweils spezifische Aufgaben: `positioning` ermöglicht die intuitive relative Positionierung von Elementen mit Befehlen wie `right=3cm of klassenname`, während `calc` komplexe Koordinatenberechnungen für präzise Platzierung von Beschriftungen ermöglicht. Die `shapes.multipart`-Bibliothek stellt das essenzielle `rectangle split`-Feature zur Verfügung, das unsere UML-Klassen in die drei charakteristischen Bereiche unterteilt. Besonders wichtig ist `arrows.meta`, da diese Bibliothek moderne Pfeilspitzen wie `Triangle[open]` für Vererbung und `Diamond[open]` für Aggregation bereitstellt, die den UML-Standards entsprechen. Die `fit`-Bibliothek wird für komplexere Layouts benötigt, falls Sie später Diagrammteile gruppieren oder umrahmen möchten.
Der zentrale Baustein unserer UML-Diagramme ist der `umlclass`-Stil, der das mächtige `rectangle split`-Feature von TikZ nutzt. Dieses Feature teilt automatisch ein Rechteck in drei gleichmäßige Bereiche auf, die den klassischen UML-Aufbau mit Klassenname, Attributen und Methoden widerspiegeln. Durch die präzise Kontrolle der Textausrichtung wird der Klassenname zentriert dargestellt, während Attribute und Methoden linksbündig ausgerichtet werden, was der UML-Konvention entspricht.

Besonders praktisch ist die Anpassbarkeit der Klassenbreite über den Parameter `text width`. Während der Standardwert von 4cm für die meisten Anwendungsfälle ausreicht, können Sie diese Breite individuell anpassen – beispielsweise auf 3cm für kompakte Diagramme oder 6cm für Klassen mit längeren Methodennamen, wie im ersten Beispiel gezeigt. Die Mindesthöhe von 2cm sorgt dafür, dass auch Klassen mit wenigen Inhalten ein ausgewogenes Erscheinungsbild behalten.
Für die verschiedenen UML-Beziehungstypen definieren wir spezialisierte Stile, die den UML-Standards entsprechen. Der `inherit`-Stil erzeugt Vererbungspfeile mit der charakteristischen dreieckigen Pfeilspitze, während `compose` und `aggregate` die unterschiedlichen Kompositions- und Aggregationsbeziehungen mit gefüllten oder leeren Diamanten darstellen. Einfache Assoziationen werden durch den `associate`-Stil als gerade Linien visualisiert, und Abhängigkeitsbeziehungen erhalten durch den `depend`-Stil ihre typische gestrichelte Darstellung.

```
% UML-Klassen-Stil
\tikzstyle{umlclass} = [
  rectangle split,
  rectangle split parts=3,
  rectangle split part align={center,left,left},
  draw,
  thick,
  text width=4cm,
  minimum height=2cm
]

% UML-Beziehungsstile
\tikzstyle{inherit} = [-{Triangle[open]}, thick]    %Vererbung
\tikzstyle{compose} = [-{Diamond}, thick]           %Komposition
\tikzstyle{aggregate} = [-{Diamond[open]}, thick]   %Aggregation
\tikzstyle{associate} = [-, thick]                  %Assoziation
\tikzstyle{depend} = [-, thick, dashed]             %Abhaengigkeit
```

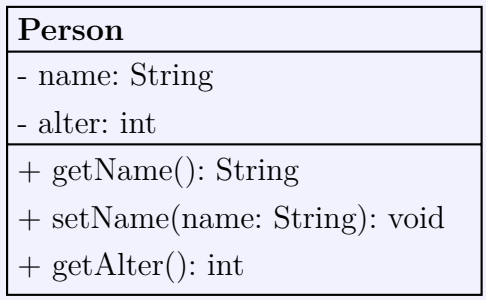
Diese Stil-Definitionen sollten in der Präambel Ihres Dokuments platziert werden und bilden das Fundament für alle nachfolgenden UML-Diagramme.

2 Einfache UML-Klasse erstellen

Mit dem definierten Stil wird die Klassenerstellung deutlich einfacher:

```
\begin{tikzpicture}
  \node[umlclass, text width=6cm] (person) {
    \textbf{Person}
    \nodepart{second}
    - name: String \\
    - alter: int
    \nodepart{third}
    + getName(): String \\
    + setName(name: String): void \\
    + getAlter(): int
  };
\end{tikzpicture}
```

Ausgabe



Die `\nodepart`-Befehle wechseln zwischen den drei Bereichen der Klasse. Innerhalb jedes Bereichs können Sie mit `\\` wie gewohnt Zeilenumbrüche einfügen, um mehrere Attribute oder Methoden untereinander aufzulisten.

3 Beziehungen zwischen Klassen

UML-Klassendiagramme leben von den Beziehungen zwischen den einzelnen Klassen. Diese Beziehungen visualisieren die Interaktionen zwischen Klassen, einschließlich Vererbung und Komposition. Ti*k*Z bietet durch unsere definierten Stile eine einfache Möglichkeit, alle wichtigen UML-Beziehungstypen professionell darzustellen.

3.1 Vererbung

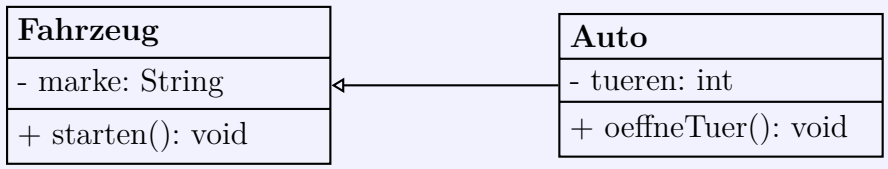
Vererbungsbeziehungen stellen dar, dass eine Klasse die Eigenschaften einer anderen Klasse übernimmt. Wir verwenden dafür den `inherit`-Stil mit seinem charakteristischen Dreieckspfeil:

```
\begin{tikzpicture}
  \node[umlclass] (fahrzeug) {
    \textbf{Fahrzeug}
    \nodepart{second}
    - marke: String
    \nodepart{third}
    + starten(): void
  };

  \node[umlclass, right=3cm of fahrzeug] (auto) {
    \textbf{Auto}
    \nodepart{second}
    - tueren: int
    \nodepart{third}
    + oeffneTuer(): void
  };

  \draw[inherit] (auto) -- (fahrzeug);
\end{tikzpicture}
```

Ausgabe



3.2 Assoziation

Einfache Assoziationen verbinden Klassen mit einer geraden Linie:

```
\begin{tikzpicture}
  \node[umlclass] (kunde) {
    \textbf{Kunde}
    \nodepart{second}
    - name: String
    \nodepart{third}
    + bestellen(): void
  };
  \node[umlclass, right=3cm of kunde] (bestellung) {
    \textbf{Bestellung}
    \nodepart{second}
    - datum: Date
    \nodepart{third}
    + bestaetigen(): void
  };
  \draw[associate] (kunde) -- (bestellung);
  \node[above] at ($ (kunde.east)!0.5!(bestellung.west) $) {bestellt};
\end{tikzpicture}
```

Ausgabe



3.3 Komposition

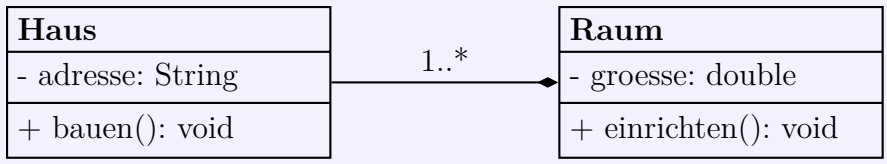
Komposition zeigt eine starke „Teil-von“-Beziehung mit gefülltem Diamanten:

```
\begin{tikzpicture}
  \node[umlclass] (haus) {
    \textbf{Haus}
    \nodepart{second}
    - adresse: String
    \nodepart{third}
    + bauen(): void
  };

  \node[umlclass, right=3cm of haus] (raum) {
    \textbf{Raum}
    \nodepart{second}
    - groesse: double
    \nodepart{third}
    + einrichten(): void
  };
```

```
  \draw[compose] (haus) -- (raum);
  \node[above] at ($ (haus.east)!0.5!(raum.west) $) {1..*};
\end{tikzpicture}
```

Ausgabe



3.4 Aggregation

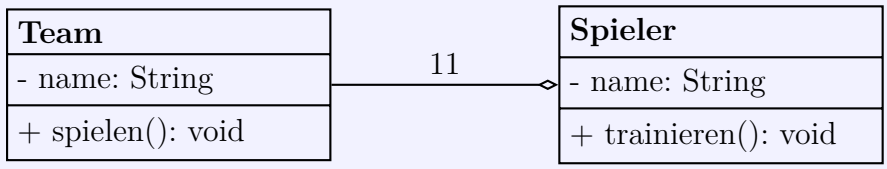
Aggregation zeigt eine schwächere „Teil-von“-Beziehung mit leerem Diamanten:

```
\begin{tikzpicture}
  \node[umlclass] (team) {
    \textbf{Team}
    \nodepart{second}
    - name: String
    \nodepart{third}
    + spielen(): void
  };

  \node[umlclass, right=3cm of team] (spieler) {
    \textbf{Spieler}
    \nodepart{second}
    - name: String
    \nodepart{third}
    + trainieren(): void
  };

  \draw[aggregate] (team) -- (spieler);
  \node[above] at ($ (team.east)!0.5!(spieler.west) $) {11};
\end{tikzpicture}
```

Ausgabe



3.5 Abhängigkeit

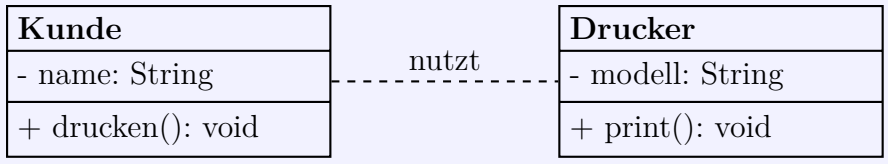
Abhängigkeiten zeigen, dass eine Klasse eine andere temporär nutzt. Der `depend`-Stil erzeugt gestrichelte Pfeile für diese Beziehung:

```
\begin{tikzpicture}
  \node[umlclass] (kunde) {
    \textbf{Kunde}
    \nodepart{second}
    - name: String
    \nodepart{third}
    + drucken(): void
  };
```

```
  \node[umlclass, right=3cm of kunde] (drucker) {
    \textbf{Drucker}
    \nodepart{second}
    - modell: String
    \nodepart{third}
    + print(): void
  };

  \draw[depend] (kunde) -- (drucker);
  \node[above] at ($ (kunde.east)!0.5!(drucker.west) $) {nutzt};
\end{tikzpicture}
```

Ausgabe



4 Eigene Befehle definieren

Für noch effizientere Diagrammerstellung können Sie eigene LaTeX-Befehle definieren. Diese Befehlsdefinitionen müssen ebenfalls in der Präambel Ihres Dokuments stehen, damit sie im gesamten Dokument verfügbar sind:

```
% Befehl fuer einfache UML-Klasse - gehoert in die Praeambel!
\newcommand{\umlklasse}[4]{
  \node[umlclass] (#1) {
    \textbf{#2}
    \nodepart{second}
    #3
    \nodepart{third}
    #4
  };
}
```

Verwendet werden kann dieser Befehl dann beispielsweise wie folgt:

```
\umlklasse{personid}{Person}{- name: String}{+ getName(): String}
```

5 Komplexes Beispiel: Schulverwaltungssystem

Hier ist ein Beispiel für ein vollständiges UML-Diagramm mit verschiedenen Beziehungstypen:

```
\begin{tikzpicture}[node distance=2.5cm]
  \node[umlclass] (person) {
    \textbf{Person}
    \nodepart{second}
    \# name: String \\
    \# geburtsdatum: Date
    \nodepart{third}
    + getName(): String \\
    + getAlter(): int
  };
```

```
  \node[umlclass, below left=of person] (student) {
    \textbf{Student}
    \nodepart{second}
    - matrikelnr: int \\
    - noten: List
    \nodepart{third}
    + einschreiben(): void \\
    + noteHinzufuegen(): void
  };

  \node[umlclass, below right=of person] (lehrer) {
    \textbf{Lehrer}
    \nodepart{second}
    - personalNr: int \\
    - gehalt: double
    \nodepart{third}
    + unterrichten(): void \\
    + notenVergeben(): void
  };

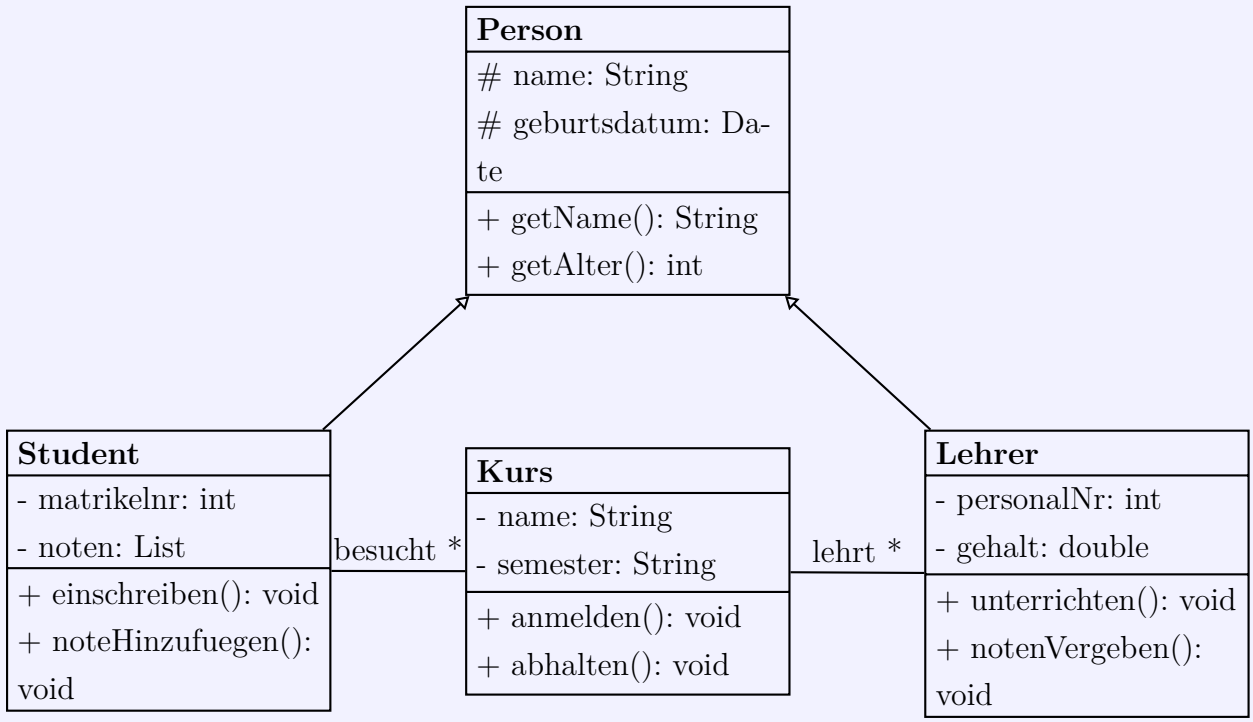
  \node[umlclass, below=2cm of person] (kurs) {
    \textbf{Kurs}
    \nodepart{second}
    - name: String \\
    - semester: String
    \nodepart{third}
    + anmelden(): void \\
    + abhalten(): void
  };

  % Vererbungspfeile
  \draw[inherit] (student) -- (person);
  \draw[inherit] (lehrer) -- (person);

  % Assoziationen
  \draw[associate] (student) -- (kurs);
  \draw[associate] (lehrer) -- (kurs);

  % Beschriftungen
  \node[above] at ($ (student)!0.5!(kurs) $) {besucht *};
  \node[above] at ($ (lehrer)!0.5!(kurs) $) {lehrt *};
\end{tikzpicture}
```

Ausgabe



6 Gruppierungen mit der `fit`-Bibliothek

Die `fit`-Bibliothek ermöglicht es, mehrere UML-Klassen zu gruppieren und mit einem gemeinsamen Rahmen zu versehen. Dies ist besonders nützlich für Pakete, Subsysteme oder logische Gruppierungen in größeren Diagrammen. Mit dem `fit`-Befehl können Sie automatisch einen Rahmen um mehrere Klassen zeichnen:

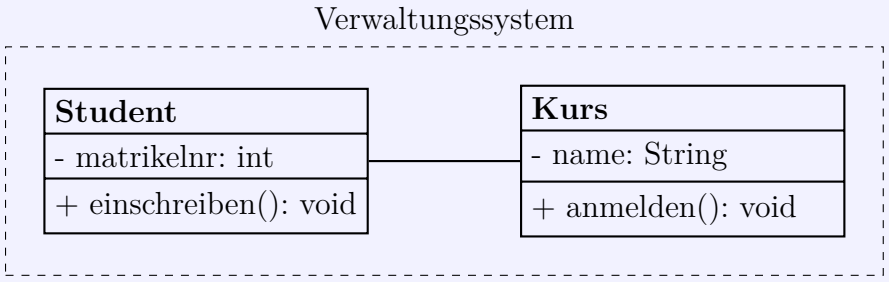
```
\begin{tikzpicture}
  \node[umlclass] (student) {
    \textbf{Student}
    \nodepart{second}
    - matrikelnr: int
    \nodepart{third}
    + einschreiben(): void
  };

  \node[umlclass, right=2cm of student] (kurs) {
    \textbf{Kurs}
    \nodepart{second}
    - name: String
    \nodepart{third}
    + anmelden(): void
  };

  % Gruppierung um beide Klassen
  \node[draw, dashed, fit=(student) (kurs),  inner sep=0.5cm, label=above:Verwaltungssystem] {};

  \draw[associate] (student) -- (kurs);
\end{tikzpicture}
```

Ausgabe



7 Zusammenfassung

Mit Ti*k*Z und benutzerdefinierten Stilen lassen sich wartbare UML-Klassendiagramme professionell erstellen. Die einmalige Definition von Stilen für Klassen und Beziehungen ermöglicht es, konsistente und ansprechende Diagramme effizient zu entwickeln.

Mit den hier vorgestellten Techniken können Sie zuverlässig professionelle UML-Diagramme für Ihren Informatikunterricht erstellen und dabei von der Flexibilität und Mächtigkeit von Ti*k*Z profitieren. Die systematische Verwendung von Stilen und eigenen Befehlen macht die Diagrammerstellung effizient und das Ergebnis professionell.